



Pashov Audit Group

Regnum Aurum veRAAC Security Review

March 21st 2026 - March 28th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Regnum Aurum veRAAC	4
5. Executive Summary	4
6. Findings	5
High findings	6
[H-01] LastProcessedDistributionIndex can skip unfinished older distributions	6
[H-02] Duplicate deduction of expired locks at epoch boundary	8
[H-03] Global epoch index corruption from <code>subtractFromEpochEndLocked</code> without check	10
Medium findings	13
[M-01] Expired epoch accounting is broken in <code>checkpointRecomputeFromLocks</code>	13
[M-02] Historical zero-power distribution prevents reward cursor advancement for users	16
Low findings	19
[L-01] <code>Claimable()</code> can overstate rewards relative to <code>claimReward()</code>	19
[L-02] Multiple reward and fee paths use <code>transfer</code> instead of <code>safeTransfer</code>	19
[L-03] Ghost voting power due to rounding error in expired locks	20



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Regnum Aurum veRAAC

Regnum Aurum veRAAC is a vote-escrow governance system that allows RAAC token holders to lock their tokens for up to four years in exchange for non-transferable veRAAC, which grants decaying voting power proportional to the remaining lock duration. The system supports lock creation, extension, and early exit through ragequit, with checkpoint-based historical voting power tracking to facilitate on-chain governance.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/contracts** repository was done by Pashov Audit Group, during which **h2134**, **jesjupyter**, **montecristo**, **BRDNS**, **pontifex** engaged to review **Regnum Aurum veRAAC**. A total of **8** issues were uncovered.

Protocol Summary

Project Name	Regnum Aurum veRAAC
Protocol Type	Vote-escrow governance
Timeline	March 21st 2026 - March 28th 2026

Review commit hash:

- [ed73117f7ce78041ad23a2239b8a581b246d1899](#)
(RegnumAurumAcquisitionCorp/contracts)

Fixes review commit hash:

- [1da61a2302a14886f43a064875f5cb88c792e99c](#)
(RegnumAurumAcquisitionCorp/contracts)

Scope

`VeRAACToken.sol``VeRAACTokenStorage.sol``LockManager.sol``PowerCheckpoint.sol``RagequitLib.sol`



6. Findings

Findings count

Severity	Amount
High	3
Medium	2
Low	3
Total findings	8

Summary of findings

ID	Title	Severity	Status
[H-01]	LastProcessedDistributionIndex can skip unfinished older distributions	High	Resolved
[H-02]	Duplicate deduction of expired locks at epoch boundary	High	Resolved
[H-03]	Global epoch index corruption from <code>subtractFromEpochEndLocked</code> without check	High	Resolved
[M-01]	Expired epoch accounting is broken in <code>checkpointRecomputeFromLocks</code>	Medium	Resolved
[M-02]	Historical zero-power distribution prevents reward cursor advancement for users	Medium	Resolved
[L-01]	Claimable() can overstate rewards relative to <code>claimReward()</code>	Low	Resolved
[L-02]	Multiple reward and fee paths use transfer instead of <code>safeTransfer</code>	Low	Resolved
[L-03]	Ghost voting power due to rounding error in expired locks	Low	Resolved



High findings

[H-01] LastProcessedDistributionIndex can skip unfinished older distributions

Severity

Impact: Medium

Likelihood: High

Description

`_updateReward()` advances `lastProcessedDistributionIndex` not by the length of the fully processed prefix of the distribution queue, but by the number of distributions inside the scanned window that happen to be fully completed.

The function scans a window starting at `lastProcessedDistributionIndex` :

```
uint256 totalDistributions = rData.distributions.length;
uint256 startIndex = uData.lastProcessedDistributionIndex;
uint256 processDistributions = startIndex + maxDistributions;
if (processDistributions > totalDistributions) processDistributions = totalDistributions;

uint256 processed;
```

Inside the loop, it increments `processed` whenever an individual distribution is already fully claimed:

```
uint40 epochsClaimed = uData.distToEligible[i].epochsClaimed;
uint40 totalEpochs = rData.distributions[i].epochs;
if (epochsClaimed >= totalEpochs) {
    processed += 1;
    continue;
}
```

At the end, the cursor is advanced by that count:

```
uData.lastProcessedDistributionIndex = uint32(uData.lastProcessedDistributionIndex +
processed);
```

This is unsafe because `processed` is not the size of a contiguous completed prefix. It is only the number of completed distributions encountered anywhere in the scanned window.

Distribution completion is time-dependent and distribution-specific, based on `epochs` and `startTimestamp` :



```
function _getClaimableAmount(uint256 eligibleRewards, uint40 epochs, uint32 startTimestamp,
uint40 cachedEpochsClaimed) internal view returns (uint256 claimableAmount, uint40
epochsClaimed){
    uint40 WEEK = 7 days;
    uint40 currentEpoch = uint40(((block.timestamp - startTimestamp)/WEEK) + 1);
    if (currentEpoch >= epochs) currentEpoch = epochs;
    uint256 claimablePerEpoch = eligibleRewards / epochs;
    for (uint40 i = cachedEpochsClaimed; i < currentEpoch; i++){
        claimableAmount += claimablePerEpoch;
        epochsClaimed++;
    }
    return (claimableAmount, cachedEpochsClaimed + epochsClaimed);
}
```

As a result, a later short-lived distribution can finish earlier than an older long-lived distribution in the same window. When that happens, the later completed distribution increments `processed`, and the final cursor move can push `lastProcessedDistributionIndex` past an older unfinished distribution.

Once that happens, the older distribution is left behind the cursor even though it is not fully claimed.

This is especially dangerous because both `_updateReward()` and `claimable()` start scanning from `lastProcessedDistributionIndex`:

```
uint256 startIndex = uData.lastProcessedDistributionIndex;
uint256 processDistributions = startIndex + maxDistributions;
if (processDistributions > totalDistributions) processDistributions = totalDistributions;
```

and in the view path:

```
uint40 epochsClaimed = uData.distToEligible[i].epochsClaimed;
uint40 totalEpochs = rData.distributions[i].epochs;
if (epochsClaimed >= totalEpochs) continue;
```

So once an unfinished older distribution is pushed behind the cursor, it can be skipped in future reward updates and future claimable calculations.

This breaks the streaming rewards model: completion of later short distributions can advance the global per-user cursor past earlier long distributions that are still in progress, creating a risk of permanent reward skipping and reward loss.

Concrete example: D0 has 4 epochs, D1 has 1 epoch. At T+1 days both are partially payable; D1 does not yet complete. At T+8, D0 pays epoch 2/4; D1 is now complete (1/1) → processed becomes 1, cursor advances to index 1. At T+15, loop starts at index 1; D0 at index 0 is permanently skipped. User loses D0 epochs 3 and 4 (50% of total rewards from that distribution).

This scenario is realistic because `distributeRewards()` accepts an arbitrary epochs parameter per distribution, so distributions can have heterogeneous stream lengths by design. A long governance-epoch stream followed by a short ragequit-penalty stream is a natural case that triggers the bug.



Recommendations

Only advance `lastProcessedDistributionIndex` by the length of the contiguous completed prefix starting from the current cursor.

In practice, this means:

- scan from `startIndex` forward,
- stop prefix advancement at the first unfinished distribution,
- do not let the completion of later distributions move the cursor past earlier unfinished ones.

More generally, the cursor should preserve the invariant that all distributions before `lastProcessedDistributionIndex` are fully processed for that user.

[H-02] Duplicate deduction of expired locks at epoch boundary

Severity

Impact: High

Likelihood: Medium

Description

The `_decayBias` function in `LockManager` calculates voting power decay by iteratively reducing `effectiveTotalLocked` at each epoch boundary via `_decreaseEffectiveTotalLocked`. However, when a checkpoint is recorded exactly at an epoch boundary (e.g., `block.timestamp` equals `epochEnd`), subsequent calls to `_decayBias` will reprocess the same boundary.

```
function _decayBias(LockState storage _lockState, int128 bias, uint256 totalLocked,
uint40 startTimestamp, uint40 endTimestamp, uint40 maxTime, address user) internal view
returns (int128 decayedBias, int128 remainder, uint256 endTotalLocked) {
    // ...

    uint40 currentTimestamp = startTimestamp;
    @> uint40 startTimestampToWeek = _roundUpToWeekInternal(startTimestamp, WEEK);
    @> uint40 epochEnd = startTimestampToWeek;
    uint256 effectiveTotalLocked = totalLocked;
    decayedBias = bias;

    while(epochEnd <= endTimestamp){
        uint40 duration = epochEnd - currentTimestamp;
        uint256 decayAmount = effectiveTotalLocked * duration/maxTime;
        if (decayAmount > MAX_CASTABLE_INT128) revert ValueTooLargeToCast();
        decayedBias -= int128(uint128(decayAmount));
    @> effectiveTotalLocked = _decreaseEffectiveTotalLocked(_lockState,user,
effectiveTotalLocked, epochEnd);
        currentTimestamp = epochEnd;
        epochEnd += 7 days;
    }
```



```
}  
  
// ...  
}
```

`_decreaseEffectiveTotalLocked` also reads `userTotalLockedAtEpochEnd[user][epochEnd]` in the user path (when user is not equal to `address(0)`), so the double deduction affects both per-user voting power calculations and global calculations.

Vulnerability Path

Timeline:

- T0: User creates multiple locks, one of which is expiring at epoch boundary E1
- Mapping `totalLockedAtEpochEnd[E1]` records the lock amount (e.g., 100)
- E1: User interacts (e.g., `create/increase` / `extend` / `withdraw`)
- `startTimestamp` = T0 (last checkpoint)
- `endTimestamp` = E1
- `_roundUpToWeekInternal(T0)` → E1 (rounds up to boundary)
- `duration` = E1 - T0 (normal decay calculation)
- Executes `_decreaseEffectiveTotalLocked(E1)`, first deduction of expired locks (100)
- Executes `_checkpointExpiredLocksCumulative`: removes E1 from `globalEpochEnds` array but fails to zero `totalLockedAtEpochEnd[E1]` mapping (residual value remains 100)
- New checkpoint records: `p.ts` = E1
- T2 (T2 > E1): User interacts again
- `startTimestamp` = E1 (last checkpoint time)
- `endTimestamp` = T2
- `_roundUpToWeekInternal(E1)` → E1 (boundary returns itself)
- Loop begins: `epochEnd` = E1, `currentTimestamp` = E1
- `duration` = E1 - E1 = 0 (no decay)
- Executes `_decreaseEffectiveTotalLocked(E1)` again, reads residual value 100 from mapping, second deduction of the same locks
- Then `epochEnd += 7 days` proceeds to the next week

Result: `effectiveTotalLocked` is deducted twice (total 200), causing subsequent decay calculations to use an artificially reduced base and inflate the user's voting power.



Impact

Users retain inflated voting power as decay calculations use an artificially reduced `effectiveTotalLocked` base. In extreme cases, repeated interactions at epoch boundaries could reduce the effective base to zero while locks remain active, granting non-decaying voting power.

Recommendations

In `_checkpointExpiredLocksCumulative`, zero out the mapping value when an epoch is processed and removed. Alternatively, skip the epoch-bucket subtraction entirely when duration is equal to 0: `if (duration > 0) { effectiveTotalLocked = _decreaseEffectiveTotalLocked(...); }`. This prevents the zero-elapsed-time iteration from deducting any expired amounts, regardless of whether the mapping was zeroed.

[H-03] Global epoch index corruption from `subtractFromEpochEndLocked` without check

Severity

Impact: Medium

Likelihood: High

Description

`_subtractFromEpochEndLocked()` removes `epochEnd` from `globalEpochEnds` after subtracting a single user's amount, without verifying that the global aggregate `totalLockedAtEpochEnd[epochEnd]` has reached zero.

This allows a single user to remove an epoch from the global index even when other users still have non-zero locked balances at the same `epochEnd`.

The subtraction logic updates the global aggregate correctly, but the index removal is unconditional:

```
uint256 currentAmountGlobal = lockState.totalLockedAtEpochEnd[epochEnd];
...
lockState.totalLockedAtEpochEnd[epochEnd] =
    currentAmountGlobal > amount ? currentAmountGlobal - amount : 0;

epochEnds = lockState.globalEpochEnds;
for (uint256 i; i < epochEnds.length; i++){
    if(epochEnds[i] == epochEnd) {
        lockState.globalEpochEnds[i] = epochEnds[epochEnds.length-1];
        lockState.globalEpochEnds.pop();
    }
}
```



At the same time, global accounting relies entirely on `globalEpochEnds` as the index of all epochs with non-zero locked balances. For example,

`_checkpointExpiredLocksCumulative(address(0), ...)` iterates over this array:

```
epochEnds = lockState.globalEpochEnds;
for (uint256 i; i < epochEnds.length; i++){
    epochEnd = epochEnds[i];
    totalLocked = lockState.totalLockedAtEpochEnd[epochEnd]

    - lockState.ragequitLockIgnore[epochEnd];

    if (totalLocked > 0 && epochEnd <= block.timestamp){
        cumulativeExpired += totalLocked;
        continue;
    }
    ...
}
```

If an epoch is removed from `globalEpochEnds` while `totalLockedAtEpochEnd[epochEnd] > 0`, that epoch becomes invisible to all subsequent global accounting.

As a result:

- existing locked amounts are no longer included in `_checkpointExpiredLocksCumulative(address(0), ...)`;
- `cumulativeExpiredGlobal` becomes understated;
- global checkpoints (`gNew.totalLocked` , `gNew.bias` , `gNew.accExpiredLocks`) are written with incorrect values;
- the error persists in checkpoint history and affects all future calculations.

This breaks core protocol invariants, including:

- global total locked accounting,
- voting power / bias decay model,
- any mechanisms relying on global checkpoints (e.g., governance, rewards, emissions).

The issue has a cross-user effect: actions of a single user can corrupt the global accounting of all other users.

The issue is reachable through standard flows that call `_subtractFromEpochEndLocked()`, including:

- `extendLock()`
- `withdrawSpecificLock()`
- `emergencyWithdrawAll()`

If repeated across multiple epochs, `globalEpochEnds` can become incomplete or nearly empty, causing the protocol to stop accounting for a significant portion of locked liquidity.



The consequence propagates into reward accounting: once a shared epoch is removed from `globalEpochEnds`, `effectiveTotalLocked` passed into `_decayBias()` becomes overstated, global bias decays faster than intended, `totalSupply()` can become lower than the sum of user voting powers, and reward distributions that use `totalSupplyAt(snapshotBlock)` as the denominator can overpay early claimers while underpaying late claimers.

Note that removing from `userEpochEnds[user]` is always correct: a user's entry is only subtracted when the user fully exits that epoch, so `userTotalLockedAtEpochEnd[user][epochEnd]` reaches zero as expected. The bug is exclusive to `globalEpochEnds`, where other users may still hold non-zero balances at the same epoch end.

Recommendations

Only remove `epochEnd` from `globalEpochEnds` when the updated `totalLockedAtEpochEnd[epochEnd]` equals zero.

The removal condition must be based on the post-update global aggregate, not on the fact that a subtraction occurred. This preserves the invariant that `globalEpochEnds` contains all epochs with non-zero global locked balances and prevents cross-user corruption of the global index.

The exact condition to guard removal is:

```
if (lockState.totalLockedAtEpochEnd[epochEnd] == 0) { globalEpochEnds[i] = epochEnds[epochEnds.length - 1]; globalEpochEnds.pop(); }
```



Medium findings

[M-01] Expired epoch accounting is broken in checkpointRecomputeFromLocks

Severity

Impact: High

Likelihood: Low

Description

During a specific lock ragequit, all unexpired lock amounts are removed from `totalLocks` and added to `ragequitLockIgnore`.

However, in ragequit finalization, not all locks are added back to `totalLocks`, since `_checkpointRecomputeFromLocks` skips expired locks at the time of finalization.

LockManager.sol

```
433:         for (uint256 i = uLockState.nextUnlockIndex; i < locks.length; i++) {
434:             uint112 amount = locks[i].amount;
435:             if (amount == 0) continue;
436:             uint40 unlockTime = locks[i].unlockTime;
437: @>             if (unlockTime <= ts) continue;
438:
439:             int128 bias;
440:             uint40 duration = unlockTime - ts;
441:             if (duration > maxTime) duration = maxTime;
442:             uint256 biasUint = uint256(amount) * uint256(duration) / uint256(maxTime);
443:             if (biasUint > MAX_CASTABLE_INT128) revert ValueTooLargeToCast();
444:             bias = int128(int256(biasUint));
445:             if (ts <= unlockTime) lockState.ragequitLockIgnore[unlockTime] -= amount;
446:             totalBias += bias;
447:             totalLocked += amount;
448:         }
```

As a result, the user lock state's `totalLocked` and `ragequitLockIgnore` are not correctly updated for locks that expired during ragequit finalization:

- `cumulativeExpired` still counts expired locks, but these amounts are not added back to `totalLocked`. As a result, `totalLocked - cumulativeExpired` will underflow.
- `totalLocked` is 0 if the single expired lock is the only lock the user has. In this case, `withdraw` will revert with `NoTokensLocked` error.

The stale ragequitLockIgnore also inflates global decay calculations. In `_decreaseEffectiveTotalLocked` (LockManager.sol L956-958), for global path (`user == address(0)`), the return value is: `effectiveTotalLocked + ragequitLockIgnore[epochEnd]` -



totalLockedAtEpochEnd[epochEnd]. When ragequitLockIgnore[epochEnd] remains non-zero for an expired lock that was never cleared, this adds a phantom amount back into effectiveTotalLocked, causing the global effective total locked base to be inflated and global voting power to decay prematurely.

In addition, _checkpointExpiredLocksCumulative(address(0)) uses ragequitLockIgnore as a suppressor: totalLocked = totalLockedAtEpochEnd[epochEnd] - ragequitLockIgnore[epochEnd]. A stale ragequitLockIgnore entry causes the expired amount for that epoch to be understated in global expired-lock accounting (cumulativeExpiredGlobal), leading to systematically incorrect global checkpoints for the affected epoch end.

POC

Scenario is like the following:

- User locks at epoch 1
- User locks at epoch 2
- User ragequits locks at epoch 1
- User finalizes ragequits
- User tries to withdraw expired lock at epoch 2 but the transaction reverts

```
diff --git a/test/unit/core/tokens/veRAACToken.test.js b/test/unit/core/tokens/veRAACToken.test.js
index 3f28ef9..3a23494 100644

--- a/test/unit/core/tokens/veRAACToken.test.js
+++ b/test/unit/core/tokens/veRAACToken.test.js
@@ -210,6 +210,40 @@ describe("veRAACTokenV2", () => {
     expect(after - before).to.equal(amount);
   });

+  it.only("reverts withdrawing a lock that expired during ragequit cooldown for a different lock", async () => {
+    const user = users[0];
+    const amountA = ethers.parseEther("100");
+    const amountB = ethers.parseEther("200");
+
+    await veRAACToken.connect(user).lock(amountA, 1);
+    await veRAACToken.connect(user).lock(amountB, 2);
+
+    const firstLockEnd = await veRAACToken.getMinLockEnd(user.address);
+    expect(firstLockEnd).to.be.gt(0n);
+
+    await time.increaseTo(firstLockEnd - 1n);
+    await ethers.provider.send("evm_mine", []);
+
+    const secondLockEnd = await veRAACToken.getMaxLockEnd(user.address);
+    expect(secondLockEnd).to.be.gt(firstLockEnd);
+
+    await expect(veRAACToken.connect(user).ragequitLock(firstLockEnd))
+      .to.emit(veRAACToken, "RagequitLockInitiated");
+  });
+}
```



```

+         const pendingRequest = await veRAACToken.ragequitRequests(user.address);
+         expect(pendingRequest.amount).to.be.gt(0n);
+         expect(pendingRequest.readyAt).to.equal(firstLockEnd + BigInt(EPOCH) - 1n);
+
+         await time.increaseTo(secondLockEnd + 1n);
+         await ethers.provider.send("evm_mine", []);
+
+         await expect(veRAACToken.connect(user).finalizeRagequit(user.address))
+             .to.emit(veRAACToken, "RagequitFinalized");
+
+         // expect(await veRAACToken.lockedBalanceOf(user.address)).to.equal(0n); // <--
+         this will underflow
+         await veRAACToken.connect(user).withdraw() // <-- this will revert with
+         NoTokensLocked() error
+     });
+
+     it("creates locks across all epochs up to the maximum, validates checkpoints and
+     totals", async function () {
+         this.timeout(1000000);
+         const user = users[0];

```

Recommendations

- In order to fix underflow, we need to add expired lock amounts to `totalLocked`.
- However, expired locks should not increase `totalBias`.
- Expired amounts should be removed from `ragequitLockIgnore`.

After all, we can consider the following fix:

```

diff --git a/contracts/libraries/governance/LockManager.sol b/contracts/libraries/governance/
LockManager.sol
index a8eda97..d991284 100644

--- a/contracts/libraries/governance/LockManager.sol
+++ b/contracts/libraries/governance/LockManager.sol
@@ -434,16 +434,15 @@ library LockManager {
     uint112 amount = locks[i].amount;
     if (amount == 0) continue;
     uint40 unlockTime = locks[i].unlockTime;

-    if (unlockTime <= ts) continue;
-
-    int128 bias;
-    uint40 duration = unlockTime - ts;
-    if (duration > maxTime) duration = maxTime;
-    uint256 biasUint = uint256(amount) * uint256(duration) / uint256(maxTime);
-    if (biasUint > MAX_CASTABLE_INT128) revert ValueTooLargeToCast();
-    bias = int128(int256(biasUint));
-    if (ts <= unlockTime) lockState.ragequitLockIgnore[unlockTime] -= amount;
-    totalBias += bias;
+    if (unlockTime > ts) {
+        uint40 duration = unlockTime - ts;
+        if (duration > maxTime) duration = maxTime;
+        uint256 biasUint = uint256(amount) * uint256(duration) / uint256(maxTime);
+        if (biasUint > MAX_CASTABLE_INT128) revert ValueTooLargeToCast();

```



```
+         totalBias += int128(int256(biasUint));  
+     }  
+  
+     lockState.ragequitLockIgnore[unlockTime] -= amount;  
+     totalLocked += amount;  
+ }
```

[M-02] Historical zero-power distribution prevents reward cursor advancement for users

Severity

Impact: Medium

Likelihood: Medium

Description

`_updateReward()` advances `lastProcessedDistributionIndex` only for distributions that enter the `epochsClaimed >= totalEpochs` branch. Other scanned entries do not move the cursor, including historical distributions where the user has no voting power at `dist.blockNumber`.

Excluded-address distributions (`dist.excludedAddress == _user`) are also permanently ineligible and exhibit the same cursor stagnation: the loop executes `continue` without incrementing `processed`, so `lastProcessedDistributionIndex` never advances past the excluded entry. This is an additional triggering condition beyond zero voting power.

The relevant logic is:

```
uint256 totalDistributions = rData.distributions.length;  
uint256 startIndex = uData.lastProcessedDistributionIndex;  
uint256 processDistributions = startIndex + maxDistributions;  
if (processDistributions > totalDistributions) processDistributions = totalDistributions;  
  
uint256 processed;  
uint256 userTotalClaimed;  
for (uint256 i = startIndex; i < processDistributions; i++) {  
    Distribution memory dist = rData.distributions[i];  
    if (dist.excludedAddress == _user) continue;  
  
    // Get user's voting power at this distribution block  
    uint256 userPowerAtDistribution = balanceOfAt(_user, dist.blockNumber);  
    if (userPowerAtDistribution > 0 && dist.totalVotingPower > 0) {  
        uint40 epochsClaimed = uData.distToEligible[i].epochsClaimed;  
        uint40 totalEpochs = rData.distributions[i].epochs;  
        if (epochsClaimed >= totalEpochs) {processed +=1;  
        continue;}  
        uint256 distAmount18 = _scaleTo18(dist.amount, tokenDecimals);  
        uint256 userShare = (userPowerAtDistribution * distAmount18) / dist.totalVotingPower;  
        // Calculate user's share of this distribution  
        if (!uData.distToEligible[i].shareUpdated){
```



```
        if (userShare > distAmount18) userShare = distAmount18;
        uData.distToEligible[i].eligibleRewards = userShare;
        uData.distToEligible[i].shareUpdated = true;}
        if (userShare == 0) continue;
        uint256 claimableAmount;
        (claimableAmount ,uData.distToEligible[i].epochsClaimed) =
        _getClaimableAmount(uData.distToEligible[i].eligibleRewards, totalEpochs,
        rData.distributions[i].startTimestamp, epochsClaimed);
        if (claimableAmount > distAmount18) claimableAmount = distAmount18;
        _claimReward(claimableAmount, _token, uData, rData, i, msg.sender);
        userTotalClaimed += claimableAmount;
    }
}

if (userTotalClaimed == 0) revert InvalidAmount();

// Mark all processed distributions
uData.lastProcessedDistributionIndex = uint32(uData.lastProcessedDistributionIndex +
processed);
```

This means that distributions with `userPowerAtDistribution == 0` are repeatedly scanned but never removed from the head of the queue.

This is especially harmful for late-joining users. If a user enters after a long reward history already exists, then a large prefix of old distributions may satisfy:

- `balanceOfAt(_user, dist.blockNumber) == 0` , and
- therefore they are never claimable by that user.

However, those entries also never increment `processed` , so they remain in front of the cursor forever.

Because processing is bounded to the window `[lastProcessedDistributionIndex, lastProcessedDistributionIndex + maxDistributions)` , this stale prefix can consume the entire processing budget and prevent the user from reaching newer distributions that are actually claimable.

The same cursor/window pattern is used by `claimable()` :

```
uint256 totalDistributions = rData.distributions.length;
uint256 startIndex = uData.lastProcessedDistributionIndex;
uint256 processDistributions = startIndex + maxDistributions;
if (processDistributions > totalDistributions) processDistributions = totalDistributions;

uint256 totalClaimable;
for (uint256 i = startIndex; i < processDistributions; i++) {
    Distribution memory dist = rData.distributions[i];
    if(dist.excludedAddress == _user) continue;

    // Get user's voting power at this distribution block
    uint256 userPowerAtDistribution = balanceOfAt(_user, dist.blockNumber);

    if (userPowerAtDistribution > 0 && dist.totalVotingPower > 0) {
```



```
uint40 epochsClaimed = uData.distToEligible[i].epochsClaimed;  
uint40 totalEpochs = rData.distributions[i].epochs;  
if (epochsClaimed >= totalEpochs) continue;
```

As a result, historically irrelevant distributions do not age out for that user and instead become persistent head-of-line blockers. This can severely degrade reward availability and, in long histories, practically starve late joiners from accessing newer rewards.

In addition to reward starvation, the stagnating cursor creates a gas denial-of-service risk for late joiners: every `claimReward()` and `claimable()` call re-traverses the same prefix of ineligible distributions. As the protocol history grows, the required `maxDistributions` parameter to reach the first eligible distribution grows linearly, eventually exceeding practical gas limits and permanently blocking any reward claim.

Recommendations

Advance `lastProcessedDistributionIndex` based on entries that are permanently irrelevant for the user, not only on distributions that are fully claimed.

In particular, distributions should be retrievable from the user's head cursor when it is known they can never yield rewards for that user, such as when `userPowerAtDistribution` == 0.

More generally, the cursor should preserve the invariant that old non-claimable head entries cannot indefinitely consume the bounded processing window and block access to newer claimable rewards.



Low findings

[L-01] Claimable() can overstate rewards relative to claimReward()

Description

`claimable()` can return a larger amount than `claimReward()` actually pays out. The issue is that `claimable()` aggregates claimable amounts in the 18-decimal internal model and only rounds once at the end, while the payout path effectively rounds down per distribution before transferring the reward.

As a result, the preview path and the payout path use different rounding semantics:

- `claimable()` may show a higher total claimable amount;
- `claimReward()` may pay slightly less in practice.

This creates a correctness issue for integrations and frontends, since the displayed claimable amount is not a reliable estimate of the actual payout.

Recommendation:

Make the view path and payout path use the same rounding model. The safest fix is to mirror the per-distribution rounding behavior in `claimable()`, so the preview matches the amount that can actually be claimed.

[L-02] Multiple reward and fee paths use transfer instead of safeTransfer

Description

The contract imports and uses `SafeERC20`, but several code paths still call raw `transfer()`:

```
raac.transfer(treasuryAddress, toTreasury);
IERC20(_rewardToken).transfer(treasuryAddress, distributable);
IERC20(_rewardToken).transfer(treasuryAddress, amountToTreasury);
IERC20(address(raac)).transfer(treasuryAddress, pendingReward);
```

USDT is a concrete example of a reward token that would fail: its `transfer()` does not return a boolean and reverts on failure in some deployments, causing the `distributeRewards` zero-VP redirect and `finalizeRemoveRewardToken` flows to revert silently or incompletely.

Impact

For standard ERC-20s this is usually fine, but non-standard reward tokens can revert or behave unexpectedly. That can break reward redirection and reward-token removal flows.



Recommendation

Use `safeTransfer()` consistently for both RAAC and arbitrary reward tokens.

[L-03] Ghost voting power due to rounding error in expired locks

Description

In `_decayBias`, when the query timestamp equals the final epoch boundary (`currentTimestamp == endTimeStamp`), the function returns early without checking if `effectiveTotalLocked` has reached zero. Due to Solidity's floor division in weekly decay calculations, a small residual bias (1 to `maxLockEpochs - 1`) can remain even after all underlying locks have expired and been removed from `effectiveTotalLocked`. This causes users to retain non-zero voting power despite having no active locked tokens.

[LockManager.sol#L912](#):

```
if (currentTimestamp == endTimeStamp) return (decayedBias < 0 ? (int128(0), remainder, effectiveTotalLocked) : (decayedBias, remainder, effectiveTotalLocked));
```

Example: User locks `10e18 + 3` (satisfying `minLockAmount`) for 4 weeks. At expiration (Week 4):

- Weekly decay: `floor((10e18 + 3) / 4) = 2.5e18` (truncated)
- 4-week total decay: `10e18`
- Residual: `decayedBias = 3`, `effectiveTotalLocked = 0`
- Query returns voting power of 3 despite zero locked tokens.

The residual voting power is bounded by `maxLockEpochs - 1` and requires querying at the exact expiration timestamp. However, this breaks the invariant that zero locked tokens should yield zero voting power, potentially allowing dust-weight votes in governance or reward calculations.

Recommendation

Add a zero-check before the early return:

```
if (currentTimestamp == endTimeStamp) {
    if (effectiveTotalLocked == 0) decayedBias = 0;
    return (decayedBias < 0 ? (int128(0), remainder, effectiveTotalLocked) : (decayedBias, remainder, effectiveTotalLocked));
}
```